

DocuSign API + Workflows on Java Spring Boot

Printable Interview Cheat Sheet + Spring Boot Code Snippets

Designed for interview prep: concise talking points plus copy-friendly Java Spring Boot snippets you can discuss out loud.

1) 30-Second Summary Answer

Best opener: "In Spring Boot, I integrate DocuSign through the Java SDK, use **Authorization Code Grant** for interactive user flows and **JWT Grant** for server-to-server automation, send envelopes using **templates or composite templates**, support **embedded signing** with recipient view when users are already in the portal, and track state with **DocuSign Connect webhooks** secured by **HMAC**. If the process needs orchestration beyond signing, I use **Workflow Builder API**."

2) Architecture You Can Describe

- **Auth service:** obtains and refreshes OAuth token; injects token into ApiClient.
- **Envelope service:** creates/sends envelopes, templates, recipients, tabs.
- **Signing service:** generates recipient view URL for embedded signing.
- **Webhook controller:** receives Connect events, validates HMAC, stores payload, processes async.
- **Workflow service:** triggers and monitors Workflow Builder instances when the business process has multiple steps.
- **Persistence:** stores enveloped, status, event history, correlation IDs, timestamps.

3) Key Talking Points

- **Auth choice:** Auth Code for user-interactive flows; JWT for backend automation.
- **Envelope strategy:** Templates for repeatable packages, composite templates for flexible runtime document/data composition.
- **Tabs:** Anchor tabs when document layout can shift; X/Y tabs when layout is fixed.
- **Status tracking:** Prefer Connect webhooks instead of polling.
- **Production readiness:** HMAC validation, idempotent event handling, async processing, externalized secrets.

4) Spring Boot Code Snippets

These are intentionally interview-friendly snippets: realistic enough to discuss design choices, small enough to memorize.

pom.xml dependencies

```
<dependencies>
  <dependency>
    <groupId>com.docusign</groupId>
    <artifactId>docusign-esign-java</artifactId>
    <version>6.6.0</version>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
</dependencies>
```

application.yml

```
docusign:
  base-path: https://demo.docusign.net/restapi
  auth-server: account-d.docusign.com
  account-id: YOUR_ACCOUNT_ID
  user-id: YOUR_IMPERSONATED_USER_ID
  integration-key: YOUR_INTEGRATION_KEY
  private-key-path: classpath:keys/docusign_private.key
  callback-url: https://your-app.example.com/ds/callback
  hmac-secret: YOUR_CONNECT_HMAC_SECRET
```

DocuSignProperties.java

```
@ConfigurationProperties(prefix = "docusign")
@Data
public class DocuSignProperties {
  private String basePath;
  private String authServer;
  private String accountId;
  private String userId;
  private String integrationKey;
  private String privateKeyPath;
  private String callbackUrl;
  private String hmacSecret;
}
```

DocuSignConfig.java

```
@Configuration
@EnableConfigurationProperties(DocuSignProperties.class)
@RequiredArgsConstructor
public class DocuSignConfig {
  private final DocuSignProperties props;

  @Bean
  public ApiClient apiClient() {
    ApiClient apiClient = new ApiClient(props.getBasePath());
    apiClient.setOAuthBasePath(props.getAuthServer());
    return apiClient;
  }
}
```

JwtAuthService.java

```
@Service
@RequiredArgsConstructor
public class JwtAuthService {
  private final ApiClient apiClient;
  private final DocuSignProperties props;

  public String getAccessToken() throws IOException, ApiException {
    byte[] privateKey =
      new ClassPathResource("keys/docusign_private.key").getInputStream().readAllBytes();
```

```
List<String> scopes = List.of("signature", "impersonation");
OAuth.OAuthToken token = apiClient.requestJWTUserToken(
    props.getIntegrationKey(),
    props.getUserId(),
    scopes,
    privateKey,
    3600);

return token.getAccessToken();
}

public ApiClient authorizedClient() throws IOException, ApiException {
    apiClient.addDefaultHeader("Authorization", "Bearer " + getAccessToken());
    return apiClient;
}
}
```

EnvelopeService.java — send an envelope with a document + anchor tab

```
@Service
@RequiredArgsConstructor
public class EnvelopeService {
    private final JwtAuthService authService;
    private final DocuSignProperties props;

    public String sendEnvelope(String signerEmail, String signerName, byte[] pdfBytes)
        throws Exception {
        ApiClient client = authService.authorizedClient();
        EnvelopesApi envelopesApi = new EnvelopesApi(client);

        Document doc = new Document();
        doc.setDocumentBase64(Base64.getEncoder().encodeToString(pdfBytes));
        doc.setName("Agreement.pdf");
        doc.setFileExtension("pdf");
        doc.setDocumentId("1");

        SignHere signHere = new SignHere();
        signHere.setAnchorString("/sn1/");
        signHere.setAnchorUnits("pixels");
        signHere.setAnchorXOffset("10");
        signHere.setAnchorYOffset("-20");

        Tabs tabs = new Tabs();
        tabs.setSignHereTabs(List.of(signHere));

        Signer signer = new Signer();
        signer.setEmail(signerEmail);
        signer.setName(signerName);
        signer.setRecipientId("1");
        signer.setRoutingOrder("1");
        signer.setTabs(tabs);

        Recipients recipients = new Recipients();
        recipients.setSigners(List.of(signer));

        EnvelopeDefinition envelope = new EnvelopeDefinition();
        envelope.setEmailSubject("Please sign this agreement");
        envelope.setDocuments(List.of(doc));
        envelope.setRecipients(recipients);
        envelope.setStatus("sent");

        EnvelopeSummary summary = envelopesApi.createEnvelope(props.getAccountId(), envelope);
        return summary.getEnvelopeId();
    }
}
```

TemplateEnvelopeService.java — send from template

```
@Service
@RequiredArgsConstructor
public class TemplateEnvelopeService {
    private final JwtAuthService authService;
    private final DocuSignProperties props;

    public String sendFromTemplate(String templateId, String signerEmail, String signerName)
        throws Exception {
        ApiClient client = authService.authorizedClient();
        EnvelopesApi envelopesApi = new EnvelopesApi(client);

        TemplateRole role = new TemplateRole();
        role.setEmail(signerEmail);
        role.setName(signerName);
        role.setRoleName("signer");

        EnvelopeDefinition env = new EnvelopeDefinition();
        env.setTemplateId(templateId);
        env.setTemplateRoles(List.of(role));
        env.setStatus("sent");

        EnvelopeSummary summary = envelopesApi.createEnvelope(props.getAccountId(), env);
        return summary.getEnvelopeId();
    }
}
```

EmbeddedSigningService.java — create recipient view URL

```
@Service
@RequiredArgsConstructor
```

```

public class EmbeddedSigningService {
    private final JwtAuthService authService;
    private final DocuSignProperties props;

    public String createRecipientView(String envelopeId,
                                     String signerEmail,
                                     String signerName,
                                     String clientId) throws Exception {
        ApiClient client = authService.authorizedClient();
        EnvelopesApi envelopesApi = new EnvelopesApi(client);

        RecipientViewRequest viewRequest = new RecipientViewRequest();
        viewRequest.setReturnUrl(props.getCallbackUrl() + "?envelopeId=" + envelopeId);
        viewRequest.setAuthenticationMethod("none");
        viewRequest.setEmail(signerEmail);
        viewRequest.setUserName(signerName);
        viewRequest.setClientId(clientId);

        ViewUrl viewUrl = envelopesApi.createRecipientView(
            props.getAccountId(), envelopeId, viewRequest);

        return viewUrl.getUrl();
    }
}

```

EmbeddedSignerController.java

```
@RestController
@RequestMapping("/agreements")
@RequiredArgsConstructor
public class EmbeddedSignerController {
    private final EnvelopeService envelopeService;
    private final EmbeddedSigningService signingService;

    @PostMapping("/{contractId}/send-for-signature")
    public Map<String, String> sendAndOpen(@PathVariable String contractId) throws Exception {
        byte[] pdf = Files.readAllBytes(Path.of("sample-agreement.pdf"));
        String email = "signer@example.com";
        String name = "Ryan Wibawa";
        String clientUserId = "1001";

        String envelopeId = envelopeService.sendEnvelope(email, name, pdf);
        String signingUrl = signingService.createRecipientView(
            envelopeId, email, name, clientUserId);

        return Map.of("envelopeId", envelopeId, "signingUrl", signingUrl);
    }
}
```

ConnectWebhookController.java — verify HMAC and respond fast

```
@RestController
@RequestMapping("/webhooks/docuSign")
@RequiredArgsConstructor
public class ConnectWebhookController {
    private final DocuSignProperties props;
    private final ApplicationEventPublisher publisher;

    @PostMapping
    public ResponseEntity<Void> handleConnect(
        @RequestHeader(value = "X-DocuSign-Signature-1", required = false) String signature,
        @RequestBody byte[] rawBody) {

        String payload = new String(rawBody, StandardCharsets.UTF_8);
        if (!isValidHmac(payload, signature, props.getHmacSecret())) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
        }

        publisher.publishEvent(new DocuSignConnectEvent(this, payload));
        return ResponseEntity.ok().build();
    }

    private boolean isValidHmac(String payload, String headerValue, String secret) {
        try {
            Mac mac = Mac.getInstance("HmacSHA256");
            mac.init(new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
            byte[] digest = mac.doFinal(payload.getBytes(StandardCharsets.UTF_8));
            String computed = Base64.getEncoder().encodeToString(digest);
            return computed.equals(headerValue);
        } catch (Exception e) {
            return false;
        }
    }
}
```

Async event consumer + idempotency sketch

```
@Component
@RequiredArgsConstructor
public class ConnectEventListener {
    private final EnvelopeEventRepository repository;

    @Async
    @EventListener
    public void onWebhook(DocuSignConnectEvent event) {
        String payload = event.payload();
        String eventKey = extractUniqueEventKey(payload); // e.g., envelopeId + status + timestamp
        if (repository.existsByEventKey(eventKey)) {
            return; // idempotent guard
        }
        repository.save(new EnvelopeEvent(eventKey, payload));
        // update domain status, notify downstream systems, archive docs, etc.
    }
}
```

WorkflowBuilderClient.java — pseudo-service outline

```
@Service
public class WorkflowBuilderClient {
    public String triggerWorkflow(String workflowDefinitionId, Map<String, Object> data) {
        // In production, call the Workflow Builder API with an OAuth token
        // and pass agreement/business data needed to start the workflow.
        return "workflow-instance-id";
    }

    public void monitorWorkflow(String workflowInstanceId) {
        // Retrieve instance state, pause/resume/cancel if needed.
    }
}
```

5) Interview Notes for the Code

- **JWT auth:** mention service-to-service automation and token generation with impersonation scope.
- **Anchor tabs:** say they are more resilient than hard-coded X/Y for dynamic documents.
- **Embedded signing:** say the signer must have **clientUserId** and the URL should be generated just-in-time.
- **Webhook handler:** emphasize HMAC validation, quick 200 response, and asynchronous downstream processing.
- **Idempotency:** explain how duplicate deliveries should not duplicate business actions.

6) Red Flags to Avoid

- Do not hardcode secrets or private keys in source code; use environment variables or secure secret storage.
- Do not store embedded signing URLs.
- Do not do heavy processing synchronously inside the webhook request thread.
- Do not assume polling is better than Connect webhooks.

Prepared as a concise printable study sheet for DocuSign + Spring Boot interview prep.